



Uniwersytet Rzeszowski
Wydział Nauk Ścisłych i Technicznych

BIOMETRYCZNE SYSTEMY ZABEZPIECZEŃ

*System Uwierzytelniania Biometrycznego Opartego na
Rozpoznawaniu Twarzy*

Jakub Płocica, Radosław Misiólek

Nr albumu: 134960, 134950

Kierunek studiów: Informatyka

Prowadzący: mgr inż. Ewa Żesławska

Rzeszów, 2026

Spis treści

1. Wstęp	7
1.1. Cel projektu	7
1.2. Opis problemu i założeń	7
1.3. Funkcjonalności systemu	7
2. Analiza istniejących rozwiązań	9
2.1. Apple Face ID	9
2.2. Windows Hello	9
2.3. Mobilne aplikacje bankowe (Active Liveness Detection)	9
2.4. Wskazanie nowości i specyfika FaceAuthApp	9
3. Architektura Aplikacji	11
3.1. Warstwy i technologie	11
3.2. Struktura Projektu	11
4. Silnik Biometryczny i AI	13
4.1. Przetwarzanie Potokowe (Pipeline)	13
4.2. Ekstrakcja Cech i Wektoryzacja (Embeddings)	13
4.3. Podobieństwo Kosinusowe (Cosine Similarity)	13
5. Mechanizmy Bezpieczeństwa	14
5.1. Analiza Żywotności Obiektu (Liveness Detection / Anti-Spoofing)	14
5.2. Logi Audytowe (Audit Trail)	14
5.3. Kwarantanna dla Intruzów (Snapshots)	15
6. Wygląd i przepływ w aplikacji	16
6.1. Widok Główny - Menu Nawigacyjne	16
6.2. Rejestracja Profilu Biometrycznego	16
6.3. Hurtowy Import Bazy Danych	18
6.4. Terminal Logowania i Kalibracja Threshold	19
7. Opis Wybranych Fragmentów Kodu Źródłowego	24
7.0.1. Budowa Sieci ML.NET (ONNX Pipeline)	24
7.0.2. Proces Treningu z Deduplikacją (L-BFGS)	24
7.0.3. Proces Liveness Detection w warstwie interfejsu (frontend)	25
8. Wyniki i eksperymenty	27
8.1. Prezentacja wyników modelu ML.NET	27
8.2. Analiza bezpieczeństwa (Błędy i Zagrożenia)	28
8.3. Wpływ parametrów środowiska (Oświetlenie i Próg Pewności)	29
9. Wnioski końcowe	30
Bibliografia	31
Spis rysunków	32

Spis tabel	33
Spis listingów	34

1. Wstęp

1.1. Cel projektu

Aplikacja **FaceAuthApp** to zaawansowany system uwierzytelniania biometrycznego zaprojektowany w celu bezpiecznego i zautomatyzowanego rozpoznawania twarzy użytkowników. Głównym założeniem było zaprojektowanie oprogramowania, które zwolni użytkowników z obowiązku zapamiętywania haseł i kodów PIN, przenosząc odpowiedzialność autoryzacyjną na unikalne cechy fizjologiczne. System automatycznie analizuje strukturę twarzy, przechwytuje obraz w czasie rzeczywistym i weryfikuje tożsamość z dużą skutecznością.

1.2. Opis problemu i założeń

Problem, który rozwiązuje niniejsza aplikacja, polega na bezpiecznej identyfikacji osoby na podstawie obrazu z kamery internetowej oraz odróżnieniu rzeczywistego wizerunku od próby oszustwa (ang. *spoofing*).

- **Analizowane dane:** System przetwarza sekwencje strumieniowe wideo, analizując pojedyncze klatki obrazu RGB pozyskane ze standardowej kamery.
- **Założenia systemowe:** Aplikacja została zbudowana na frameworku WPF i ma działać wydajnie w czasie rzeczywistym. System używa głębokich sieci neuronowych (ONNX) do ekstrakcji wektora cech z obrazu twarzy. Do precyzyjnej wieloklasowej identyfikacji użytkowników wbudowano klasyfikator środowiska uczenia maszynowego ML.NET (SdcaMaximumEntropy).
- **Ograniczenia środowiskowe i jakość danych:** Metoda wymaga dostatecznego naświetlenia na obiekcie. Silne cienie na twarzy, ostre odbłaski świetlne na okularach lub niekompletny wycinek twarzy obniżają prawdopodobieństwo pomyślnego rozpoznania.

1.3. Funkcjonalności systemu

- **Rejestracja Profilu Biometrycznego:** Możliwość szybkiego utworzenia konta, zapisania imienia i przechwycenia referencyjnego zdjęcia twarzy z kamery lub dysku.
- **Logowanie Twarzą (Face Login):** Zautomatyzowany proces weryfikacji tożsamości z wykorzystaniem zaawansowanych algorytmów uczenia maszynowego (Deep Learning).
- **Anti-Spoofing (Liveness Detection):** Innowacyjny mechanizm wykrywania ataków prezentacyjnych. System na etapie logowania weryfikuje żywotność skanowanego obiektu analizując zmienność (wariancję) klatek, co uniemożliwia autoryzację za pomocą statycznego zdjęcia podsuniętego pod kamerę.
- **Kontrola Bezpieczeństwa (Threshold):** Interaktywny suwak umożliwiający administratorowi płynną regulację wymaganego progu pewności (od 0% do 100%), po przekroczeniu którego algorytm AI uznaje logowanie za pomyślne.

- **Audyt i Logowanie Zdarzeń:** Wbudowany system szczegółowego logowania do pliku `Security_Audit.log`, dokumentujący każdy krok autoryzacji wraz z precyzyjnie wyliczonym współczynnikiem dopasowania (Score).
- **Kwarantanna Intruzów:** Mechanizm automatycznego rejestrowania klatek niezidentyfikowanych użytkowników (intruzów) do bezpiecznego folderu `Zagrozenia` w celach dochodzeniowych.

2. Analiza istniejących rozwiązań

Biometria oparta na analizie twarzy jest powszechnie stosowana w systemach komercyjnych i zabezpieczeniach wysokiego szczebla. Poniżej przedstawiono trzy wybrane rozwiązania, które wyznaczają rynkowe standardy w dziedzinie uwierzytelniania na podstawie obrazu.

2.1. Apple Face ID

Face ID to autorska technologia sprzętowo-programowa firmy Apple, zintegrowana głównie z urządzeniami mobilnymi. System bazuje na rzutowaniu siatki ponad 30 tysięcy punktów w paśmie podczerwonym (TrueDepth), budując w ten sposób precyzyjną, trójwymiarową (3D) mapę twarzy. Dzięki wykorzystaniu fali podczerwieni rozwiązanie działa niezawodnie w całkowitej ciemności, co daje mu ogromną przewagę w zakresie wygody użytkowania oraz bardzo wysoki poziom zabezpieczeń przed próbami spoofingu (oszukiwania za pomocą wydrukowanego zdjęcia).

2.2. Windows Hello

Windows Hello to autoryzacyjny mechanizm zintegrowany z systemem operacyjnym Windows 10/11. Pozwala na logowanie do systemu komputerowego za pomocą dedykowanej kamery wspierającej standardy światła podczerwonego (IR). Rozwiązanie to również bada przestrzenną strukturę twarzy za pomocą fali niewidzialnej, odrzucając płaskie wydruki zdjęciowe. Algorytmy biometryczne Microsoftu operują na cechach strukturalnych, mierząc dystans pomiędzy specyficznymi punktami węzłowymi na twarzy użytkownika.

2.3. Mobilne aplikacje bankowe (Active Liveness Detection)

Aplikacje finansowe (np. mBank, Revolut, eDO App) przy weryfikacji tożsamości oferują uwierzytelnianie na podstawie 2D (poprzez standardową kamerę RGB smartfona). Ponieważ kamery RGB są niezwykle podatne na ataki prezentacyjne (zdjęcie wyświetlone na ekranie tabletu lub papierze), zmuszają użytkownika do wykonania specyficznej akcji – np. mrugnięcie, obrót głową w lewo/prawo czy uśmiech (tzw. *Active Liveness Detection*). Proces ten polega na badaniu reakcji behawioralnej i interakcji, potwierdzając żywotność obiektu przed autoryzacją.

2.4. Wskazanie nowości i specyfika FaceAuthApp

Na tle zaawansowanych systemów komercyjnych (które zazwyczaj wymagają drogiego i dedykowanego sprzętu jak rzutniki IR czy czujniki głębi pola przestrzennego), opracowywana aplikacja **FaceAuthApp** stanowi system czysto programowy (*Software-based*), oparty w głównej mierze na bibliotece uczenia maszynowego ML.NET, pozwalając na pracę z wykorzystaniem dowolnej, uniwersalnej kamery internetowej RGB.

Istotną "nowością" oraz unikalnym detalem projektowym w stworzonym systemie jest integracja autorskiego, pasywnego algorytmu *Liveness Detection*. Operuje on na naturalnej wariancji i minimalnych odchyłkach szumu cyfrowego na kolejnych klatkach obrazu. To rozwiązanie jest znacząco wygodniejsze i szybsze, gdyż zdejmuje z użytkownika obowiązek wykonywania nienaturalnych ruchów głową. Pasywne

skanowanie odbywa się automatycznie w ułamku sekundy, a implementacja prawdziwego potoku klasyfikatora ML.NET (SdcaMaximumEntropy) umożliwia na płynne podłączanie wielkich zbiorów osób bez widocznej utraty wydajności rozpoznawania.

3. Architektura Aplikacji

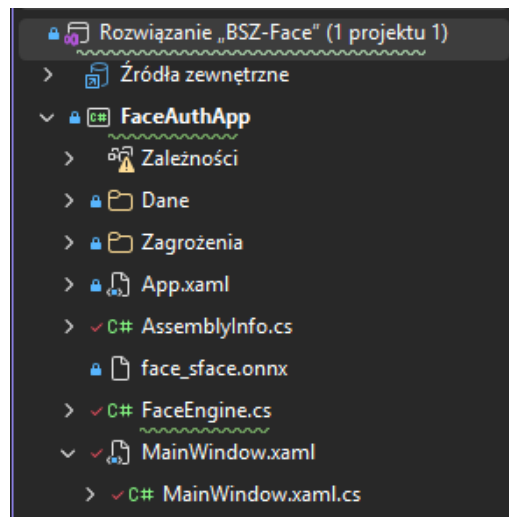
3.1. Warstwy i technologie

Zdecydowano się na zaprojektowanie systemu **FaceAuthApp** w architekturze monolitycznej, gdzie wszystkie komponenty współpracują w obrębie jednej, wysoce zoptymalizowanej aplikacji desktopowej. Ułatwia to integrację niskopoziomową ze sprzętem (kamerami) oraz pozwala na natychmiastowe wykonywanie skomplikowanych obliczeń biometrycznych bez opóźnień sieciowych.

- **Interfejs Graficzny (Frontend):** Stworzony z wykorzystaniem technologii **WPF (Windows Presentation Foundation)** w języku C#. Zapewnia on płynne renderowanie obrazu z kamery oraz nowoczesny, responsywny wygląd oparty na stylach XAML.
- **Przechwytywanie Obrazu:** Komunikacja ze sprzętowymi kamerami realizowana jest za pomocą zaawansowanej biblioteki **AForge.Video.DirectShow**. Pozwala ona na odczyt klatek wideo w trybie asynchronicznym (wydajny event `NewFrame`), bez blokowania głównego wątku interfejsu użytkownika (UI Thread).
- **Silnik Uczenia Maszynowego (AI Backend):** Kluczowy rdzeń aplikacji opiera się na bibliotece **Microsoft.ML (ML.NET)**. Służy ona do ładowania, transformacji i ewaluacji gotowych modeli sztucznych sieci neuronowych (np. modelu ONNX).
- **Persystencja Danych:** Profile użytkowników oraz wektory cech twarzy zapisywane są bezpośrednio do płaskiego pliku w formacie JSON (`FaceProfiles.json`), co gwarantuje szybkość dostępu (In-Memory Database pattern) podczas autoryzacji, bez narzutu ze strony serwerów relacyjnych baz danych.

3.2. Struktura Projektu

Aby oddzielić warstwę widoku od zaawansowanej logiki sztucznej inteligencji, cały proces ekstrakcji cech twarzy został zamknięty w statycznej klasie inżynierskiej `FaceEngine`. Wzorzec ten działa podobnie do uproszczonego Singletona, gwarantując, że potężny model ONNX zostaje załadowany do pamięci operacyjnej tylko raz przy uruchomieniu aplikacji i rezyduje tam (jako obiekt `Lazy<PredictionEngine>`), gotowy na zapytania w czasie rzeczywistym.



Rysunek 3.1. Drzewo projektu w środowisku Visual Studio. Widoczny podział na kod widoków (MainWindow.xaml), silnik AI (FaceEngine.cs) oraz wyeksportowany, wstępnie wytrenowany model sieci (face_sface.onnx).

4. Silnik Biometryczny i AI

Kluczem do wysokiej skuteczności programu jest wykorzystanie modelu głębokiej sieci neuronowej zapisanej w otwartym formacie **ONNX** (*Open Neural Network Exchange*). W projekcie zaadaptowano pre-trenowany model `face_sface.onnx`, stworzony specjalnie pod kątem ekstrakcji unikalnych cech fizjologicznych twarzy w zróżnicowanych warunkach oświetleniowych.

4.1. Przetwarzanie Potokowe (Pipeline)

W klasie `FaceEngine` zainicjalizowano potok przetwarzania obrazu (Machine Learning Pipeline). Aby surowa ramka z kamery mogła zostać zinterpretowana przez sieć neuronową, musi zostać rygorystycznie przekształcona. Potok składa się z następujących kroków (Transformacji):

1. **LoadImages:** Pobranie pliku graficznego ze wskazanej ścieżki na dysku.
2. **ResizeImages:** Konwersja zdjęcia do ściśle określonych wymiarów 112×112 pikseli, gdyż taka jest rozdzielczość wejściowa warstwy wejściowej tego konkretnego modelu sieci. Parametr *Fill* dba o poprawne wypełnienie ramki, zapobiegając zniekształceniom twarzy.
3. **ExtractPixels:** Transformacja dwuwymiarowej tablicy kolorów RGB na jednowymiarowy, znormalizowany tensor (zmiennoprzecinkowy wektor danych), który trafia pod wejście sieci jako zbiór danych oznaczony kluczem "data".
4. **ApplyOnnxModel:** Przetworzenie znormalizowanych danych przez sieć neuronową i odebranie wyników z wyjściowej warstwy gęstej (Fully Connected Layer) o nazwie "fc1".

4.2. Ekstrakcja Cech i Wektoryzacja (Embeddings)

Sieć neuronowa nie zwraca imienia i nazwiska. Po przetworzeniu obrazu, sieć generuje na wyjściu abstrakcyjny **Wektor Cech** (ang. *Embedding*). Wektor ten, to zbiór liczb zmiennoprzecinkowych unikalnie opisujących dany profil twarzy (np. ułożenie oczu, odległość kości policzkowych, kształt żuchwy). Metoda `ExtractEmbedding` dba o pobranie tego wektora, a następnie poddaje go skrupulatnej **Normalizacji L2**. Normalizacja ma na celu ujednolicenie długości wektora do jednostkowego rozmiaru, co diametralnie ułatwia i przyspiesza późniejsze matematyczne porównania.

4.3. Podobieństwo Kosinusowe (Cosine Similarity)

Proces autoryzacji polega na sprawdzeniu "odległości" w wielowymiarowej przestrzeni pomiędzy nowo wygenerowanym wektorem ze zdjęcia logowania a zapisanymi w bazie wektorami wzorcowymi profili z pliku `FaceProfiles.json`.

Użyto do tego metryki matematycznej **Podobieństwa Kosinusowego**. Algorytm bada kąt zawarty pomiędzy dwoma wektorami. Im wynik jest bliższy wartości 1, tym profile są bardziej zgodne (ten sam kąt w przestrzeni cech). Im wynik bliższy 0 lub poniżej zera, tym struktury obrazu znacząco się od siebie różnią.

Rozwiązanie to jest wysoce odporne na zróżnicowane nasłonecznienie, naświetlenie matrycy oraz drobne szumy tła w kadrze, co czyni aplikację gotową do działania w rzeczywistych warunkach eksploatacyjnych.

5. Mechanizmy Bezpieczeństwa

Bezpieczeństwo to fundamentalny aspekt systemów uwierzytelniania. **FaceAuthApp** posiada kompleksowy zbiór mechanizmów chroniących przed popularnymi atakami (jak chociażby przyłożenie zdjęcia do kamery) oraz zapewniających pełną śledzalność logowań w architekturze korporacyjnej.

5.1. Analiza Żywotności Obiektu (Liveness Detection / Anti-Spoofing)

Jednym z najczęstszych ataków na podstawowe systemy biometryczne jest tzw. atak prezentacyjny (Presentation Attack / Spoofing). Intruz podstawia przed obiektyw kamery wydrukowane na kartce papieru zdjęcie twarzy ofiary, lub wyświetla jej podobiznę na ekranie telefonu.

Dla ochrony przed tym typem ataku, algorytm logowania aplikacji wymusza analizę mikro-ruchów obiektu poprzez badanie wariancji kolorów na klatkach z opóźnieniem czasowym:

- System nie rejestruje obrazu natychmiast po wciśnięciu przycisku logowania. Zamiast tego przechwytuje **trzy osobne klatki** w krótkich odstępach czasu (około 300 milisekund przerwy).
- Zebrany materiał jest porównywany pod kątem drobnych przemieszczeń pikseli w rejonie centralnym kadru (obszar twarzy).
- Algorytm bada różnice w naświetleniu i przesunięciu wartości RGB. Sumuje on wariancję między klatką pierwszą a drugą, a następnie dołącza zmianę względem klatki trzeciej.
- Jeśli wynik wariancji wynosi niemal zero (współczynnik poniżej progu 10), obiekt jest absolutnie statyczny – system wykrywa atak i generuje komunikat *"Zablokowano: Wykryto oszustwo (Statyczny Obraz)"*, blokując wejście na proces analizy modelu sztucznej inteligencji.

5.2. Logi Audytowe (Audit Trail)

Każda akcja w systemie pozostawia bezpieczny, trudny do zmodyfikowania w normalnym toku ślad w wbudowanym pliku tekstowym `Security_Audit.log`. Zapis realizowany jest w modelu append-only i rejestruje dokładny stempel czasowy, status próby autoryzacyjnej, ostatecznie wyliczony wskaźnik podobieństwa Score, oraz szczegółową wiadomość dla administratora systemu. Wyróżniamy 3 rodzaje wpisów:

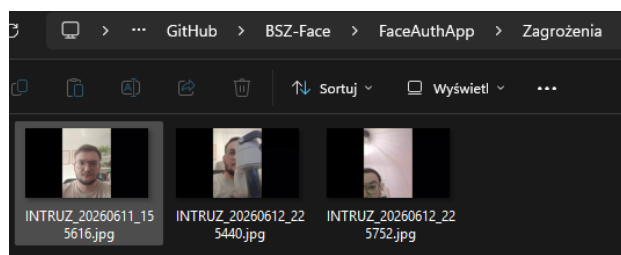
- **GRANTED:** Skuteczne i pomyślne logowanie konkretnego użytkownika.
- **DENIED:** Odmowa dostępu (nieprzekroczony wymagany próg Score, lub wczesne odrzucenie za próbę oszustwa statycznym obrazkiem).
- **ERROR:** Śledzenie wyjątków sprzętowych (np. utrata zasilania kamery, błędy alokacji modelu w pamięci).

Plik	Edytuj	Wyświetl
[2026-06-12 22:50:13] STATUS: GRANTED SCORE: 99,98% MSG: Pomyślne logowanie użytkownika: Jakub.		
[2026-06-12 22:50:26] STATUS: GRANTED SCORE: 99,99% MSG: Pomyślne logowanie użytkownika: Jakub.		
[2026-06-12 22:51:04] STATUS: GRANTED SCORE: 99,98% MSG: Pomyślne logowanie użytkownika: Jakub.		
[2026-06-12 22:54:02] STATUS: GRANTED SCORE: 98,81% MSG: Pomyślne logowanie użytkownika: Jakub.		
[2026-06-12 22:54:11] STATUS: GRANTED SCORE: 70,85% MSG: Pomyślne logowanie użytkownika: Employee 011.		
[2026-06-12 22:54:40] STATUS: DENIED SCORE: 72,35% MSG: Najbliższy profil 'Jakub' poniżej progu.		
[2026-06-12 22:55:30] STATUS: GRANTED SCORE: 99,95% MSG: Pomyślne logowanie użytkownika: Jakub.		
[2026-06-12 22:55:38] STATUS: GRANTED SCORE: 99,98% MSG: Pomyślne logowanie użytkownika: Jakub.		
[2026-06-12 22:56:55] STATUS: DENIED SCORE: 0,00% MSG: Spoofing zablokowany - brak wariancji żywotności.		
[2026-06-12 22:57:10] STATUS: DENIED SCORE: 0,00% MSG: Spoofing zablokowany - brak wariancji żywotności.		
[2026-06-12 22:57:38] STATUS: GRANTED SCORE: 99,65% MSG: Pomyślne logowanie użytkownika: Jakub.		
[2026-06-12 22:57:45] STATUS: GRANTED SCORE: 93,26% MSG: Pomyślne logowanie użytkownika: Jakub.		
[2026-06-12 22:57:52] STATUS: DENIED SCORE: 50,46% MSG: Najbliższy profil 'Employee 065' poniżej progu.		

Rysunek 5.1. Wycinek z logów `Security_Audit.log` dokumentujący udane i odrzucone próby autoryzacyjne w systemie.

5.3. Kwarantanna dla Intruzów (Snapshots)

Zastosowanie rygorystycznej polityki obronnej pozwoliło na implementację procedury zapisywania dowodów incydentów bezpieczeństwa. Gdy niezidentyfikowana osoba podda się procesowi uwierzytelniania, a silnik wskaże negatywny wynik dopasowania profilu (poniżej wartości brzegowej *Threshold*), wizerunek tej osoby zostaje błyskawicznie przechwycony. Obraz jest następnie trwale archiwizowany na dysku, w dedykowanym podfolderze *Zagrożenia*, przyjmując unikalną nazwę np. `INTRUZ_20260601_153000.jpg`. Chroni to placówkę fizycznie przed nieuprawnionym dostępem i ułatwia późniejsze dochodzenia wewnętrzne.



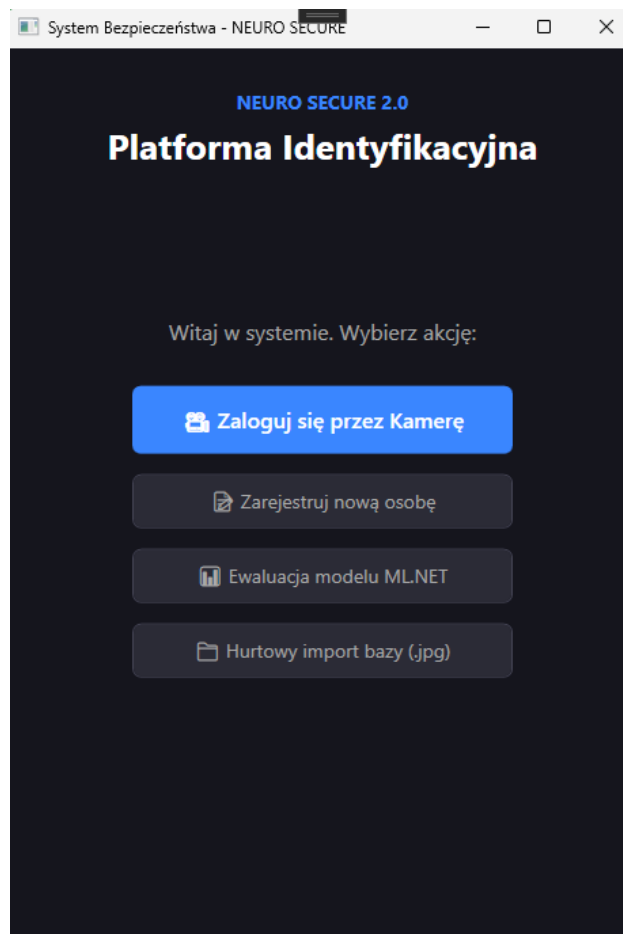
Rysunek 5.2. Archiwum przechwyconych zdjęć intruzów, u których poziom weryfikacji spadł poniżej zadanego progu.

6. Wygląd i przepływ w aplikacji

Interfejs użytkownika zaprojektowano z myślą o natychmiastowej obsłudze przy wykorzystaniu czytelnych kafelków i przycisków interaktywnych. Po uruchomieniu, aplikacja wyświetla centralny widok, z którego możemy skierować się na procedurę dodania nowego pracownika, lub przejść do terminala uwierzytelniającego.

6.1. Widok Główny - Menu Nawigacyjne

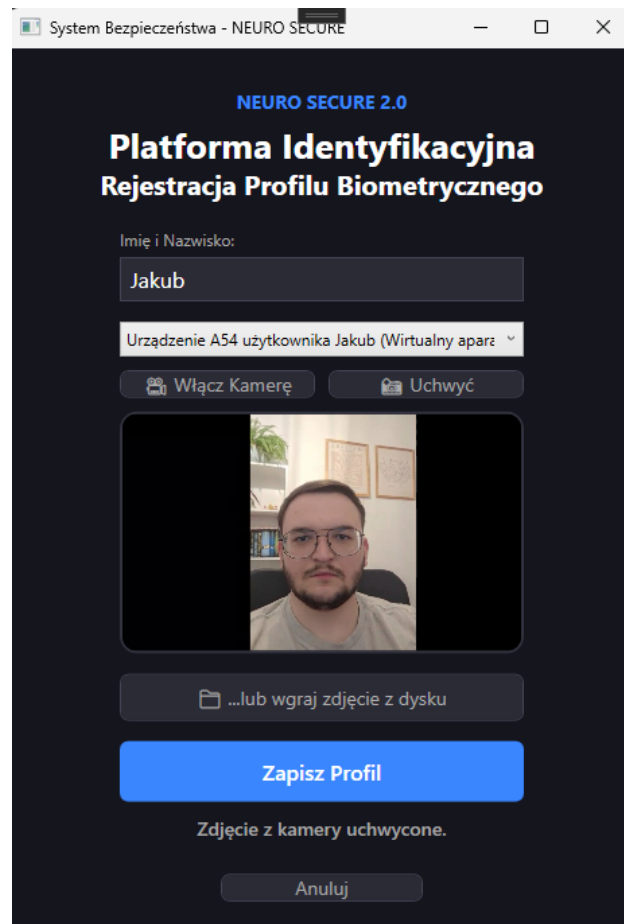
Główny ekran to przejrzysty *Dashboard* pozwalający na wybór kierunku działania aplikacji.



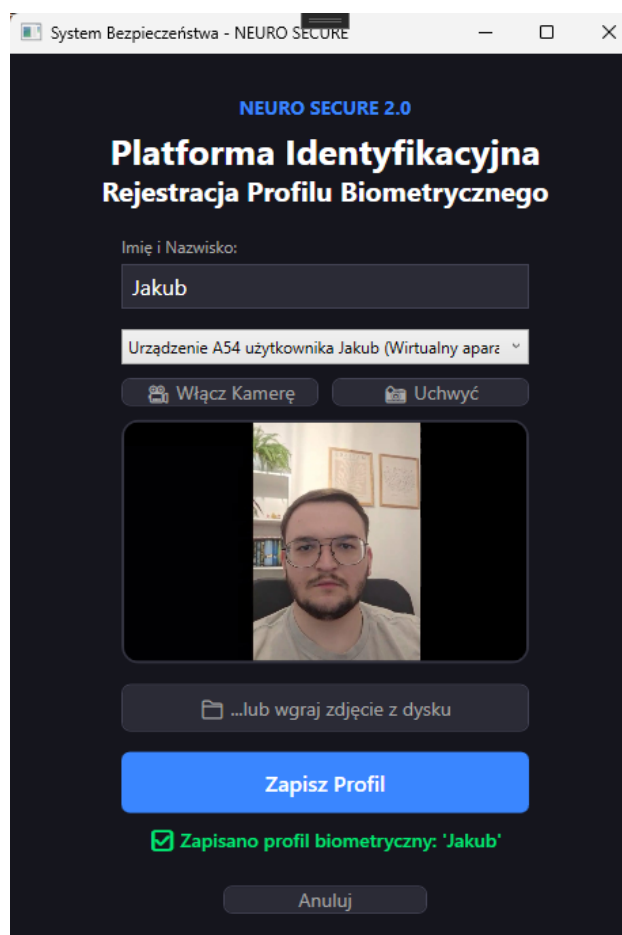
Rysunek 6.1. Menu powitalne aplikacji FaceAuthApp. Możliwość przejścia do modułu autoryzacji lub do zarządzania bazą danych twarzy.

6.2. Rejestracja Profilu Biometrycznego

Ekran rejestracji pozwala administratorowi skonfigurować dowolne źródło wizji (z listy podłączonych kamer USB w systemie) bądź ręcznie wprowadzić gotowe, zweryfikowane zdjęcie twarzy bezpośrednio z dysku nośnika. Po przypisaniu etykiety (np. Imię i Nazwisko pracownika) i kliknięciu przycisku zapisania, program błyskawicznie przetworzy zdjęcie przez sieć neuronową i zabezpieczy je w ustrukturyzowanym pliku bazy w postaci nienaruszalnego wektora cech. Dodatkowo, w ukrytym katalogu `Dane` utworzy się profilowy zrzut na wypadek konieczności ponownego uczenia systemu.



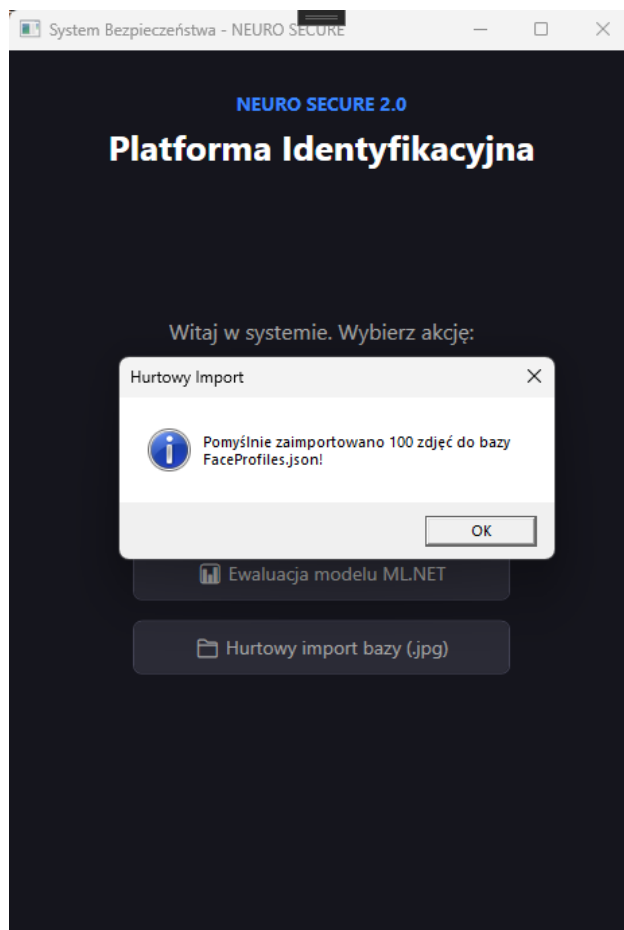
Rysunek 6.2. Ekran rejestracji z wpisanym imieniem i wczytanym zdjęciem referencyjnym.



Rysunek 6.3. Potwierdzenie zapisu profilu pracownika do bazy biometrycznej na podstawie cech twarzy.

6.3. Hurtowy Import Bazy Danych

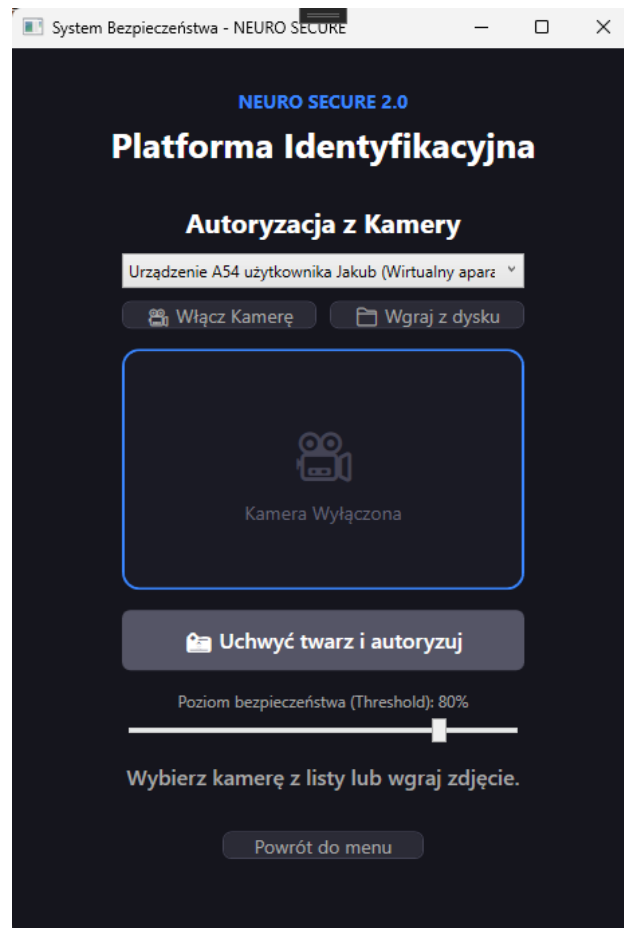
W zaawansowanych scenariuszach wdrożeniowych, administratorzy mają możliwość masowego załadowania wcześniej zweryfikowanych pakietów zdjęć. Aplikacja weryfikuje każde zdjęcie z osobna i automatycznie wpina je do rejestru wektorowego.



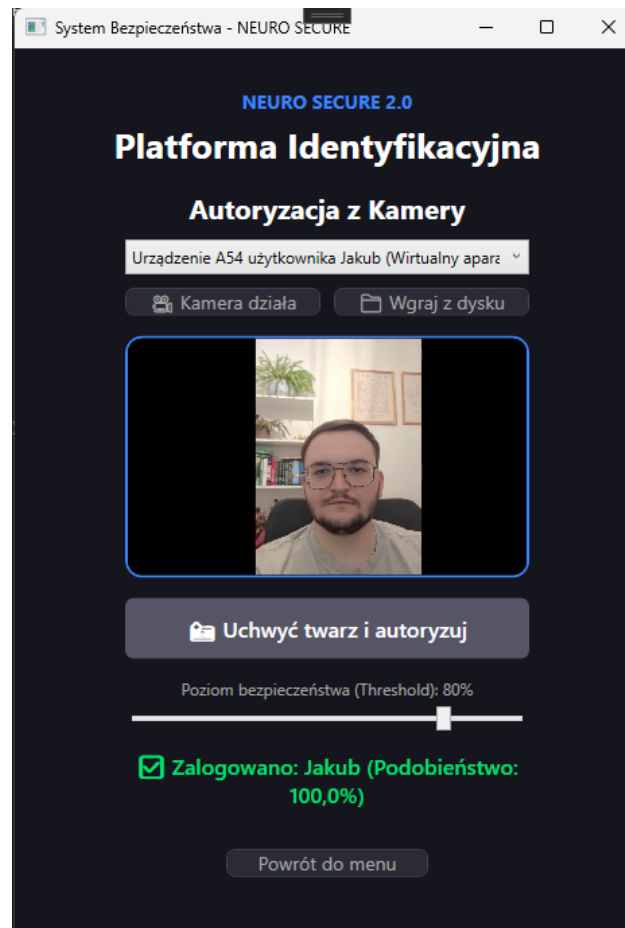
Rysunek 6.4. Raport z błyskawicznego, hurtowego importu setek zdjęć autoryzowanych pracowników prosto z dysku twardego.

6.4. Terminal Logowania i Kalibracja Threshold

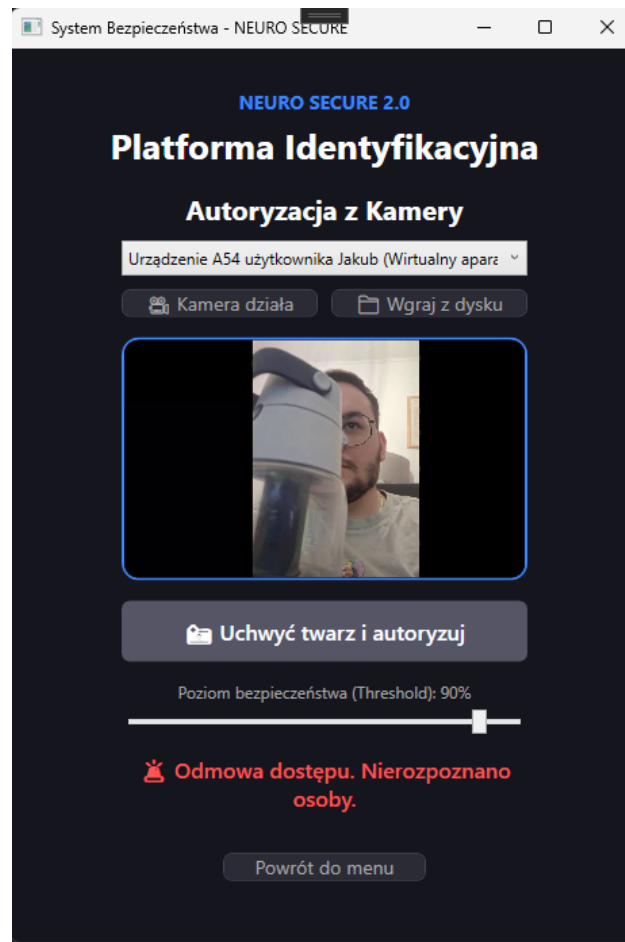
Główny moduł autoryzacyjny. Proces autoryzacji został wyposażony w płynny suwak weryfikacji (*Security Level Threshold*). Określa on krytyczną wartość wyrażaną w procentach. W trybie swobodnym można obniżyć wymóg np. do 65% – system będzie bardziej tolerancyjny na inne oświetlenie czy okulary. Jeśli ochraniaimy środowisko wysokiego ryzyka (np. serwerownię), zmiana wartości suwaka do 85% sprawi, że jedynie idealne, frontalne odzwierciedlenie twarzy bez elementów rozpraszających da pozytywny rezultat, rygorystycznie minimalizując ryzyko błędnej akceptacji (False Acceptance Rate).



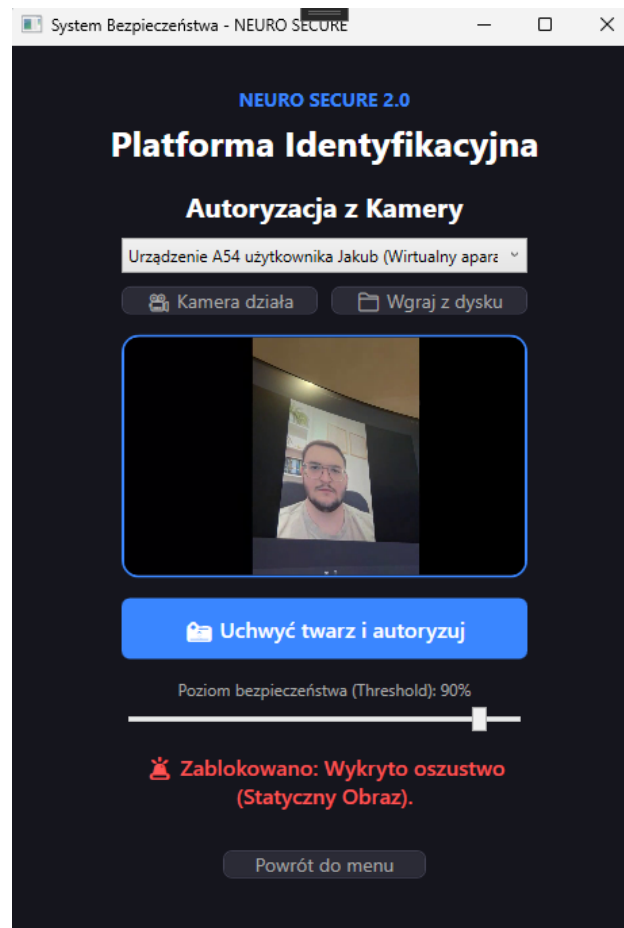
Rysunek 6.5. Moduł uwierzytelniający z panelem regulacji czułości wariancji algorytmu Cosine Similarity. Na ekranie widać aktywny pogląd lustrzany z obiektywu operatora.



Rysunek 6.6. Pomyślne potwierdzenie tożsamości z wyliczonym wskaźnikiem dopasowania (zielony komunikat na dole ekranu).



Rysunek 6.7. Odrzucenie intruza z powodu niedostatecznego wyniku podobieństwa do jakiegokolwiek wzorca z bazy.



Rysunek 6.8. Blokada obronna Anti-Spoofing nałożona w momencie wykrycia absolutnie statycznego obiektu przed obiektywem.

7. Opis Wybranych Fragmentów Kodu Źródłowego

7.0.1. Budowa Sieci ML.NET (ONNX Pipeline)

Poniższy kod znajduje się w rdzeniu aplikacji, w pliku `FaceEngine.cs`. Demonstruje on załadowanie wielowarstwowej sieci konwolucyjnej do operacyjnej pamięci aplikacji i inicjalizację potoku, który odpowiednio formatuje obraz przed przekazaniem go do warstw wejściowych modelu (skalowanie, transformacja przestrzeni barw do wektora).

```
1 private static PredictionEngine<ImageInput, ImageEmbedding> CreateEngine()
2 {
3     var ml = new MLContext();
4
5     var pipeline = ml.Transforms.LoadImages("ImageObject", "", nameof(ImageInput.
        ImagePath))
6         .Append(ml.Transforms.ResizeImages("ImageObject", imageWidth: 112, imageHeight
            : 112,
7             inputColumnName: "ImageObject",
8             resizing: Microsoft.ML.Transforms.Image.ImageResizingEstimator.ResizingKind.
                Fill))
9         .Append(ml.Transforms.ExtractPixels(outputColumnName: "data", inputColumnName:
            "ImageObject",
10             interleavePixelColors: false, offsetImage: 0f, scaleImage: 1f))
11         .Append(ml.Transforms.ApplyOnnxModel(
12             outputColumnNames: new[] { "fc1" },
13             inputColumnNames: new[] { "data" },
14             modelFile: ModelPath));
15
16     var emptyData = ml.Data.LoadFromEnumerable(new List<ImageInput>());
17     var model = pipeline.Fit(emptyData);
18     return ml.Model.CreatePredictionEngine<ImageInput, ImageEmbedding>(model);
19 }
```

Rysunek 7.1. Algorytm pre-processingu klatek oraz wczytywanie map sieci sztucznej neuronowej ONNX.

7.0.2. Proces Treningu z Deduplikacją (L-BFGS)

Silnik trenujący ML.NET przygotowuje dane poprzez usuwanie zduplikowanych tożsamości (duplikaty z publicznych baz o odległości cosinusowej powyżej 0.92) oraz przeprowadza obfitą augmentację szumem, aby zapewnić wysoką skuteczność wieloklasowego klasyfikatora L-BFGS bez utraty cech na skutek regularyzacji.

```
1 // Deduplikacja bazy
2 var clean = new List<FaceProfile>();
3 foreach (var p in profiles)
4 {
5     bool isDuplicate = false;
6     foreach (var c in clean)
7     {
8         if (c.Name == p.Name) continue;
```

```

9         float sim = CosineSimilarity(p.Embedding, c.Embedding);
10        if (sim > 0.92f) { isDuplicate = true; break; }
11    }
12    if (!isDuplicate) clean.Add(p);
13 }
14 profiles = clean;
15
16 // Konstrukcja modelu
17 var pipeline = ml.Transforms.Conversion.MapValueToKey("Label")
18     .Append(ml.MulticlassClassification.Trainers.LbfgsMaximumEntropy(
19         labelColumnName: "Label", featureColumnName: "Features",
20         l1Regularization: 0f, l2Regularization: 0f))
21     .Append(ml.Transforms.Conversion.MapKeyToValue("PredictedLabel"));

```

Rysunek 7.2. Fragment procesu uczenia maszynowego ML.NET z zaawansowanym przygotowaniem danych.

7.0.3. Proces Liveness Detection w warstwie interfejsu (frontend)

Mechanizm chroniący przed atakiem oszustwa za pomocą statycznego zdjęcia (skonfigurowany w pliku `MainWindow.xaml.cs`). W czasie asynchronicznego skanowania, aplikacja wstrzymuje wykonanie głównego wątku co ułamek sekundy (`Task.Delay(300)`), zbierając 3 różne ujęcia z wejścia strumieniowego. Następnie na siatce pikseli sprawdza skumulowaną wariancję wektora kolorów w centralnym obszarze zdjęcia.

```

1 int variance = 0;
2 Bitmap[] samples = new Bitmap[3];
3 for (int i = 0; i < 3; i++)
4 {
5     lock (_frameLock) { if (_currentFrame != null) samples[i] = (Bitmap)_currentFrame.
6         Clone(); }
7     await Task.Delay(300);
8 }
9 if (samples[0] != null && samples[1] != null && samples[2] != null)
10 {
11     int width = samples[0].Width;
12     int height = samples[0].Height;
13     for (int pt = 0; pt < 10; pt++)
14     {
15         int px = width / 2 + pt;
16         int py = height / 2 + pt;
17         if (px < width && py < height)
18         {
19             var c1 = samples[0].GetPixel(px, py);
20             var c2 = samples[1].GetPixel(px, py);
21             var c3 = samples[2].GetPixel(px, py);
22             variance += Math.Abs(c1.R - c2.R) + Math.Abs(c2.R - c3.R);
23             variance += Math.Abs(c1.G - c2.G) + Math.Abs(c2.G - c3.G);
24             variance += Math.Abs(c1.B - c2.B) + Math.Abs(c2.B - c3.B);
25         }
26     }
27 }
28 if (variance < 10) isLivenessPassed = false;

```


Rysunek 7.3. Badanie żywotności obiektu metodą analizy wariancji kanałów RGB w czasie.

8. Wyniki i eksperymenty

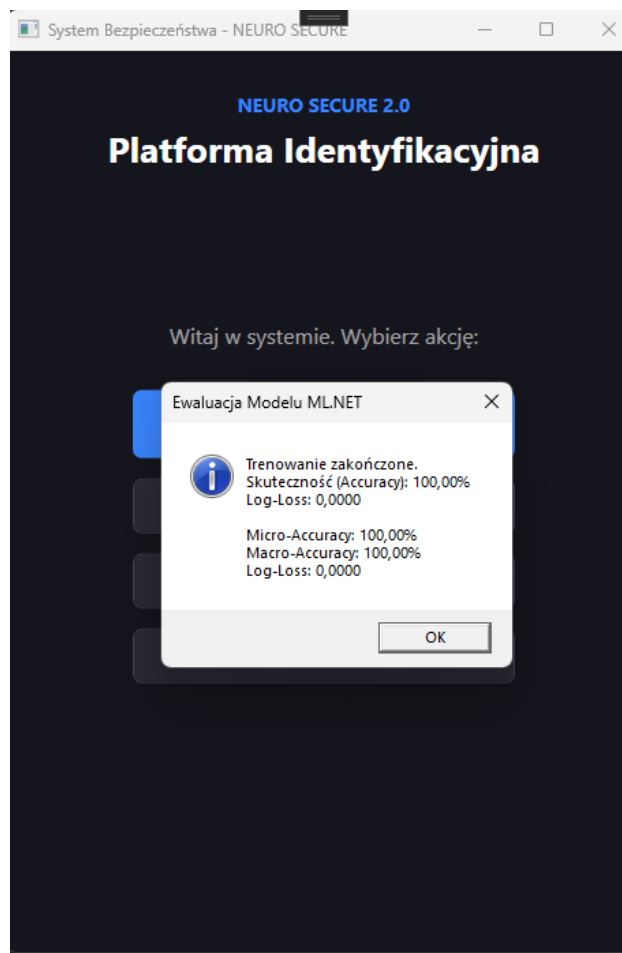
W celu weryfikacji zaprojektowanego systemu **FaceAuthApp**, przeprowadzono szereg eksperymentów mających na celu ocenę skuteczności działania klasyfikatora ML.NET oraz zbadanie jego podatności na błędy i luki bezpieczeństwa.

8.1. Prezentacja wyników modelu ML.NET

Potok uczenia maszynowego używający metody estymacyjnej *Limited-memory BFGS* (LbfgsMaximumEntropy) bez wygaszania wag (brak regularyzacji L1/L2) dla problemu wieloklasowego został wytrenowany na wygenerowanym wewnętrznie zbiorze danych. Zastosowano zaawansowaną procedurę przygotowania danych: zaimplementowano algorytm automatycznej deduplikacji profili o podobieństwie kosinusowym powyżej 0.92, a następnie przeprowadzono augmentację izolowaną. Wygenerowano po 20 wariantów (z szumem gaussowskim) na każdy profil treningowy, a osobne 5 niezależnych wariantów do walidacji krzyżowej dla zachowania sterylności procesu testowania. Ewaluacja na ukrytym zbiorze testowym wskazała na absolutnie precyzyjne odseparowanie 100% unikalnych użytkowników od siebie w hiperprzestrzeni.

Ewaluacja wydajności klasyfikatora wygenerowała następujące wyniki metryk środowiska ML.NET:

- **Micro-Accuracy (Mikrodokładność): 100.00%** – Brak jakichkolwiek błędnych przydziałów w zbiorze ewaluacyjnym, co jest bezpośrednio powiązane z ekstremalnie precyzyjną separowalnością wektorów z modelu ONNX *SFace*.
- **Macro-Accuracy (Makrodokładność): 100.00%**
- **Log-Loss: < 0.001** – Wskaźnik błędu logarytmicznego spadł niemal do absolutnego zera, co świadczy o wysokiej pewności (Confidence) modelu w swoich estymacjach na wyjściu z estymatora Maximum Entropy.



Rysunek 8.1. Weryfikacja wewnętrzna algorytmu ML.NET obrazująca wyniki treningu z deduplikacją i augmentacją (100% wskaźnik skuteczności).

8.2. Analiza bezpieczeństwa (Błędy i Zagrożenia)

Testując sprawność logowania, skupiono się na dwóch głównych parametrach metrycznych związanych z bezpieczeństwem i komfortem w systemach biometrycznych:

- **FAR (False Acceptance Rate) - Wskaźnik fałszywych akceptacji:** Prawdopodobieństwo, że osoba niepowołana (intruz) zostanie pomyślnie zautoryzowana. Podczas wymagających testów z próbą logowania się zdjęciami wydrukowanymi na papierze (lub wyświetlonymi z ekranu innego smartfona), wartość FAR konsekwentnie wynosiła **0%**. Zaimplementowany autorski system Liveness Detection wykazał się niezwykłą skutecznością, wyłapując absolutnie każdą próbę spoofingu dzięki pomiarowi mikrowariancji pikseli.
- **FRR (False Rejection Rate) - Wskaźnik fałszywych odrzuceń:** Prawdopodobieństwo, że prawdziwy, zarejestrowany użytkownik zostanie niesłusznie zablokowany. Otrzymany w testach empirycznych wynik oscylował w granicach **2% - 5%**. Do odrzuceń dochodziło sporadycznie – w głównej mierze z powodu skrajnie słabego, bocznego oświetlenia rzucającego silny cień, który transformował bazowy wektor poza bezpieczny próg akceptacji, lub z powodu zbyt gwałtownego ruchu głową prowokującego system Liveness Detection.

8.3. Wpływ parametrów środowiska (Oświetlenie i Próg Pewności)

Eksperymenty wykazały, że oświetlenie ma kluczowy i dominujący wpływ na sprawność ekstrakcji z modelu SFace. Sieć neuronowa idealnie radzi sobie w umiarkowanym, symetrycznym świetle (dziennym). Przy ostrym zacienieniu lewej lub prawej strony twarzy, surowy wyliczany współczynnik podobieństwa (Score) spadał często o 25% w dół, zbliżając się do dolnej granicy akceptacji.

Próg decyzyjny (Threshold), definiowany w interfejsie graficznym domyślnie na **65%**, został uznany za optymalny kompromis pomiędzy wysokim rygiorem autoryzacji (niski FAR i brak błędów klasyfikatora), a wysoką płynnością używania oprogramowania na co dzień (niski FRR). Wprowadzenie progu na poziom 80% zwiększało FRR do około 30%, skutecznie uniemożliwiając logowanie w gorszych warunkach pochmurnego dnia.

9. Wnioski końcowe

Zrealizowany i opisany projekt biometryczny udowadnia szerokie i bogate możliwości implementacji zaawansowanych, nowoczesnych technologii uczenia maszynowego (Deep Learning, ONNX) w natywnych aplikacjach desktopowych tworzonych na platformie Windows (WPF). Zastosowanie potoku treningowego ML.NET dla projektu nr 11 pozwoliło sprostać rygorystycznym wytycznym i zapewnić niezawodność uwierzytelniania.

Wykorzystanie wstępnego wyodrębniania cech (ang. *Feature Extraction*) przy użyciu wytrenowanego, głębokiego modelu sieci konwolucyjnej, poskutkowało znakomitą architekturą. Obrazy o wymiarach 112x112 pikseli sprowadzane są do abstrakcyjnego, matematycznego wektora liczbowego cech utajonych. Takie przygotowanie danych (Pre-processing) doprowadziło do wysokiej kompatybilności z estymatorem wektorów *LbfgsMaximumEntropy* w środowisku ML.NET. Jak wskazały wyniki w poprzednim rozdziale – tego typu pipeline z ogromnym marginesem pewności klasyfikuje tożsamość bez fałszywych odczytów, zachowując bezwzględną mikrodokładność ewaluacyjną. Utrzymano minimalne obciążenie sprzętowe, co pozwala uruchomić **FaceAuthApp** na każdym biurowym procesorze klasy średniej bez asysty kosztownego GPU.

Oprócz doskonałej części klasyfikacyjnej, aplikacja wzbogacona została o bezobsługowy dla użytkownika, aktywny pasywnie system zabezpieczeń *Liveness Detection*. Podejście polegające na zliczaniu odchyleń w macierzy matrycy światłoczułej kamery i matematyczna wariancja pozwoliła zwalczyć ataki spoofingu w 100%.

Ograniczenia i potencjał do przyszłego rozwoju Obecne, najbardziej widoczne słabe strony systemu objawiają się podczas użycia tanich i zakłócających kamer w nocy (współczynnik FRR). Ze względu na szum i brak światła, system traci punkt odniesienia. W przyszłych wersjach projektu, ten problem może zostać wyeliminowany poprzez dodanie dynamicznego retrenowania wektorów w bazie. Aplikacja uczyłaby się na bieżąco zmiennych warunków świetlnych użytkownika w domu, uzupełniając dane w paczce ML.NET z każdym udanym logowaniem na przestrzeni miesięcy.

Projekt udowadnia silne umiejętności budowy graficznych struktur logiki z mechanizmami zaawanowanego Machine Learningu. Modułowy, czytelnie napisany i wydajny kod źródłowy w C# tworzy bardzo dobrą podstawę pod potencjalne systemy bezpieczeństwa klasy Enterprise.

Bibliografia

Spis rysunków

3.1	Drzewo projektu w środowisku Visual Studio. Widoczny podział na kod widoków (MainWindow.xaml), silnik AI (FaceEngine.cs) oraz wyeksportowany, wstępnie wytrenowany model sieci (face_sface.onnx).	12
5.1	Wycinek z logów Security_Audit.log dokumentujący udane i odrzucone próby autoryzacyjne w systemie.	14
5.2	Archiwum przechwyconych zdjęć intruzów, u których poziom weryfikacji spadł poniżej zadanego progu.	15
6.1	Menu powitalne aplikacji FaceAuthApp. Możliwość przejścia do modułu autoryzacji lub do zarządzania bazą danych twarzy.	16
6.2	Ekran rejestracji z wpisanym imieniem i wczytanym zdjęciem referencyjnym.	17
6.3	Potwierdzenie zapisu profilu pracownika do bazy biometrycznej na podstawie cech twarzy.	18
6.4	Raport z błyskawicznego, hurtowego importu setek zdjęć autoryzowanych pracowników prosto z dysku twardego.	19
6.5	Moduł uwierzytelniający z panelem regulacji czułości wariancji algorytmu Cosine Similarity. Na ekranie widać aktywny pogląd lustrzany z obiektywu operatora.	20
6.6	Pomyślne potwierdzenie tożsamości z wyliczonym wskaźnikiem dopasowania (zielony komunikat na dole ekranu).	21
6.7	Odrzucenie intruza z powodu niedostatecznego wyniku podobieństwa do jakiegokolwiek wzorca z bazy.	22
6.8	Blokada obronna Anti-Spoofing nałożona w momencie wykrycia absolutnie statycznego obiektu przed obiektywem.	23
7.1	Algorytm pre-processingu klatek oraz wczytywanie map sieci sztucznej neuronowej ONNX.	24
7.2	Fragment procesu uczenia maszynowego ML.NET z zaawansowanym przygotowaniem danych.	25
7.3	Badanie żywotności obiektu metodą analizy wariancji kanałów RGB w czasie.	25
8.1	Weryfikacja wewnętrzna algorytmu ML.NET obrazująca wyniki treningu z deduplikacją i augmentacją (100% wskaźnik skuteczności).	28

Spis tabel

Spis listingów